LIVE TABLET WHITEBOARD

Production V1 — Product Requirements, Technical Architecture, Implementation Plan & Cost Model

Working concept: Turn a tablet with a stylus into a wireless, browser-based visual-thinking surface. A user draws on the tablet while another person watches the canvas live through a normal browser link. The laptop can remain dedicated to Google Meet, Zoom, Teams, coding tools, or other software.

Document purpose

This document is written so that an engineer, coding agent, product agent, or another AI system can understand what is being built, what is explicitly out of scope, how the realtime system should work, what constitutes V1 completion, and what the expected operating costs are.

Document	Production V1 Specification
Status	Build-ready product definition
Primary use case	Tablet → live shared canvas → browser viewer
Initial audience	Technical interview candidates, interviewers, students, tutors, engineers
Deployment target	Public internet / production beta
Core constraint	No installation required; browser-first
V1 philosophy	Reliable realtime canvas before AI or advanced collaboration

Current market/technical note

Existing products cover parts of this workflow, including interview whiteboards and general collaborative canvases. The purpose of V1 is therefore not to claim that realtime whiteboards are novel. The differentiation should come from the tablet-first workflow, extremely low-friction room sharing, stylus-first interaction, and later expansion into visual-thinking/AI workflows.

1. Executive Product Definition

One-sentence definition: A web application that lets a person open a room on a tablet, draw/write with a stylus, and have those drawing operations appear in near-real-time on another browser through a shareable URL.

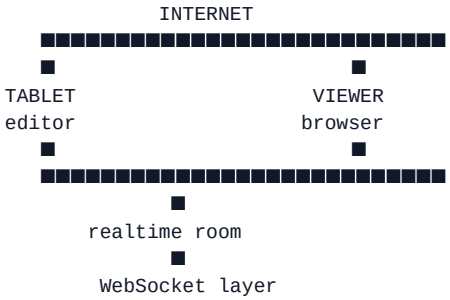
The first production release is deliberately narrow. It is not a video-conferencing platform, not a complete interview platform, and not an AI whiteboard. It is a realtime visual canvas that works alongside existing communication software.

Primary scenario

```
Candidate laptop
  ■■■ Google Meet / Zoom / Teams / coding environment

Candidate tablet
  ■■■ Browser → room URL → stylus drawing

Interviewer
  ■■■ Browser → same room URL → live read-only canvas
```



Core value proposition

- Use a tablet as a dedicated visual input device without buying or installing special hardware.
- Keep the laptop free for Meet/Zoom, code, documentation, or screen sharing.
- Share a short URL instead of asking the other person to install an application.
- Make pen strokes appear live rather than screen-sharing the entire tablet display.
- Persist the session so the user can return to it and export the canvas.

Non-goals for V1

- No built-in video or audio calling.
- No native Android/iOS app.
- No AI handwriting recognition or AI diagram cleanup in the initial release.
- No complex diagramming language or automatic architecture generation.
- No enterprise SSO, team administration, billing, or organization management.
- No requirement for the tablet and laptop to be on the same Wi-Fi network.

2. Users & Primary Workflows

Primary persona: technical interview candidate

A candidate is on a video call using a laptop. The interviewer asks a system-design, algorithm, ML-system, mathematical, or architecture question. The candidate opens the room on a tablet and explains by drawing. The interviewer receives a URL and watches the canvas in a browser.

Workflow A — Create and share

- Open the product landing page.
- Click **Create Whiteboard**.
- Server creates a random room ID and a signed room/session token.
- User is placed directly into the editor.
- User can copy a viewer URL.
- User sends the viewer URL to the interviewer.

Workflow B — Tablet pairing

- User opens the same room URL on the tablet.
- Tablet detects that it should act as an editor/device surface.
- User can optionally use a QR code shown on the laptop to avoid typing the URL.
- Stylus input begins immediately.
- No account is required for anonymous beta usage.

Workflow C — Viewer

- Interviewer opens the viewer URL.
- Viewer receives the current canvas state.
- Subsequent strokes/shapes/text operations arrive in realtime.
- Viewer is read-only by default.
- Viewer can zoom/pan and optionally follow the candidate's viewport.

Workflow D — Rejoin

- Refresh the viewer or editor.
- The room reconstructs from persisted state/snapshot plus recent operations.
- If the network temporarily disappears, the client queues safe local changes and resynchronizes when possible.

Secondary personas for later validation

- Tutor teaching mathematics, algorithms, physics, or engineering.
- Engineer explaining architecture to a remote colleague.
- Student collaborating on problem solving.
- Interview coach conducting mock interviews.
- Presenter/creator explaining a diagram while keeping the main laptop screen private.

3. Functional Requirements — V1

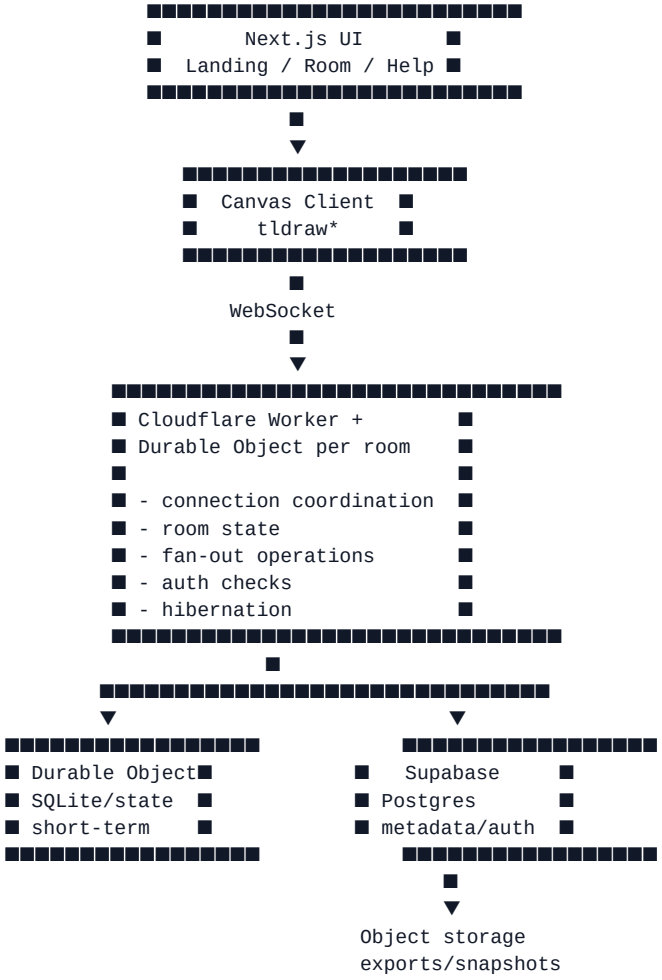
ID	Requirement	Priority	Acceptance condition
FR-01	Create room anonymously	P0	A user can create a room in <3 seconds under normal network conditions.
FR-02	Shareable viewer URL	P0	Viewer URL can be copied and opened in a second browser/device.
FR-03	Stylus drawing	P0	S Pen/Apple Pencil/touch produces smooth strokes without obvious lag.
FR-04	Realtime sync	P0	Viewer receives drawing operations with target p95 visible latency <250 ms under normal conditions.
FR-05	Canvas tools	P0	Pen, eraser, select, line/arrow, rectangle, text, undo/redo, zoom/pan.
FR-06	Read-only viewer	P0	Viewer cannot accidentally modify candidate's canvas.
FR-07	Persistence	P0	Refresh/rejoin restores the canvas for active retained rooms.
FR-08	Room security	P0	Unlisted/random room IDs are not enumerable; access tokens are validated.
FR-09	Connection status	P1	User can see Connected/Reconnecting/Offline state.
FR-10	QR pairing	P1	Laptop can display QR to open/pair the room on tablet.
FR-11	Export PNG/PDF	P1	User can export current canvas.
FR-12	Feedback	P1	User can submit feedback/bug report from app.
FR-13	Basic analytics	P1	Track room creation, successful connection, session duration, viewer join, export, error.
FR-14	Responsive UI	P0	Works on modern Chromium/Safari mobile/tablet browsers and desktop browsers.

Canvas behavior

- Pen: pressure-sensitive where supported; configurable stroke width.
- Eraser: stroke/object erasing depending on the canvas engine.
- Selection: move/resize objects.
- Shapes: rectangle, ellipse, line, arrow.
- Text: keyboard text insertion.
- Navigation: one-finger/stylus drawing; two-finger pan/zoom where browser/device behavior permits.
- Undo/redo must remain deterministic for the editor.
- Viewer should render the same logical document rather than receiving a video stream.

4. Realtime Architecture

Core principle: transmit semantic canvas operations, not pixels. A stroke is a set of points and metadata; a rectangle is an object; text is a text object. This reduces bandwidth, makes persistence possible, and allows later AI analysis of the canvas.



Suggested production stack

Layer	Recommendation	Reason
Web app	Next.js + TypeScript	Fast iteration, strong ecosystem, easy deployment.
Canvas	tldraw SDK or another production-licensed canvas engine	Avoid rebuilding drawing primitives and canvas interaction.
Realtime	Cloudflare Workers + Durable Objects + WebSockets	Room-oriented coordination, long-lived connections, geographic edge network.
Database	Supabase Postgres	Room metadata, optional users, feedback, analytics references.
Storage	Cloudflare R2 or Supabase Storage	Exports and larger artifacts.
Hosting	Vercel or Cloudflare for frontend	Simple CI/CD and HTTPS.
Analytics	PostHog/Umami or equivalent	Product usage and funnel measurement.
Monitoring	Sentry + platform logs	Frontend/runtime errors and diagnostics.

Important licensing constraint

The current tldraw SDK is source-available but its default license does not permit public production use without an appropriate license. tldraw currently offers a 100-day trial and says commercial production use requires a commercial

license; startup pricing may be available. A hobby license is for non-commercial use and includes a watermark. This must be resolved before public production launch.

If the goal is a real public product, treat the canvas-engine license as a first-class dependency rather than assuming the GitHub repository being available means the production use is free.

5. Room & Data Model

```
Room
- room_id: random, high-entropy public identifier
- editor_token_hash: server-side authorization reference
- created_at
- expires_at
- status
- last_activity_at
- owner_user_id (nullable in anonymous beta)

Session
- session_id
- room_id
- client_id
- role: editor | viewer
- joined_at
- left_at
- user_agent
- device_class

Canvas persistence
- room_id
- snapshot_version
- serialized_document / snapshot reference
- operation sequence / checkpoint metadata
- updated_at

Feedback
- feedback_id
- room_id (nullable)
- category
- rating
- message
- created_at
```

Room security model

- Use cryptographically random room IDs; never sequential integers.
- Separate editor capability from viewer capability.
- Viewer links should not grant editing privileges.
- Use short-lived signed session tokens where possible.
- Rate-limit room creation and WebSocket connection attempts.
- Do not put sensitive canvas contents into analytics events.
- Add room expiration/deletion policy for anonymous sessions.
- Use HTTPS/WSS everywhere.
- Define a clear abuse report mechanism before public launch.

Privacy posture

V1 should collect the minimum data necessary. Anonymous rooms should be ephemeral by default. If persistent accounts are introduced later, provide deletion controls and clearly document retention. Do not record interview audio/video in V1.

6. Realtime Reliability & UX Requirements

The product succeeds or fails on perceived latency. A technically correct whiteboard that feels delayed will be rejected by users.

Metric	V1 target	How to measure
Local stylus rendering	<16–30 ms perceived response	Client performance telemetry/manual testing
Editor → viewer p95	<250 ms in normal regions	Timestamp operation creation and first render
Reconnect	<5 seconds where service/network recovers	Synthetic test
Room creation	<3 seconds	Frontend telemetry
Canvas restore	<3 seconds for normal session	Measure first usable render
Availability	Target 99.9% after beta stabilizes	Uptime monitor
Data loss	No acknowledged operation lost under normal failure	Chaos/failure tests

Failure scenarios to test

- Tablet changes from Wi-Fi to mobile data.
- Laptop/browser refreshes.
- Viewer joins after the candidate has already drawn 100+ objects.
- Two viewers join simultaneously.
- WebSocket drops for 10–30 seconds.
- Two browser tabs edit the same room accidentally.
- Tablet sleeps and wakes.
- Large canvas with thousands of strokes.
- Slow 3G/4G connection.
- Server/room instance is hibernated and reactivated.

7. UX Specification

Landing page

```
LIVE WHITEBOARD

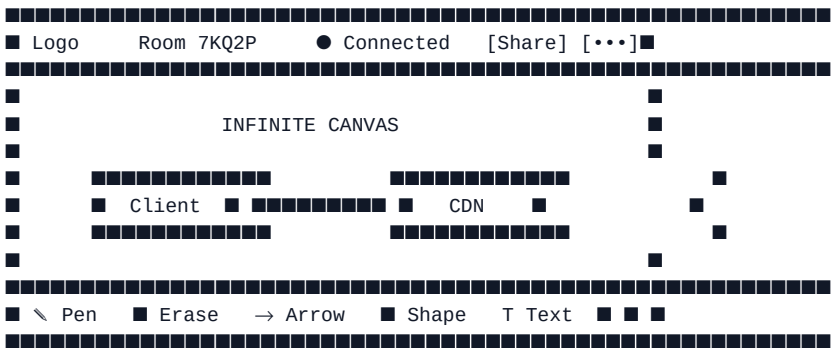
Turn your tablet into a live visual-thinking surface.

[ Create a whiteboard ]

No installation. Share one link.

Use it for:
• Technical interviews
• System design
• Teaching
• Remote explanations
• Collaborative problem solving
```

Room screen — editor



Share panel

```
Share this whiteboard

Viewer link
https://app.example.com/r/7KQ2P?mode=view

[ Copy link ]

[ Show QR code ]

Viewer permissions: View only
Editor device: This browser
```

Viewer screen

- Minimal chrome; canvas is dominant.
- Show candidate/editor name only if provided.
- Display LIVE/RECONNECTING status.
- Viewer cannot modify content in V1.
- Optional Follow Candidate button.

8. Implementation Plan — Production Beta

Phase	Estimated time	Deliverables
0. Validation	1–2 days	Confirm tablet browser/stylus behavior, choose canvas engine/license, test WebSocket architecture.
1. Foundation	2–3 days	Repo, CI/CD, Next.js app, domain, environments, basic room creation.
2. Canvas	2–4 days	Canvas integration, stylus input, tools, responsive tablet layout.
3. Realtime	3–5 days	Room server, WebSockets, operation sync, reconnect, presence/status.
4. Persistence	2–3 days	Snapshots/checkpoints, room restore, expiration policy.
5. Sharing	1–2 days	Viewer URLs, QR pairing, permissions.
6. Production hardening	3–5 days	Rate limits, error handling, observability, security, tests.
7. Feedback/analytics	1–2 days	Feedback form, product analytics, basic admin view.
8. Beta QA	3–5 days	Real device tests: Samsung tablet/S Pen, iPad/Apple Pencil, desktop Chrome/Safari/Firefox.

Expected solo-developer schedule

A focused developer could reach a usable public beta in roughly 2–4 weeks, depending heavily on familiarity with the selected canvas engine and realtime stack. A more conservative 4–6 week schedule is appropriate if the objective is a polished, reliable public launch rather than a demo.

Definition of done

- A new visitor can create a room without an account.
- A Samsung/Android tablet with S Pen can draw smoothly in the browser.
- A second browser can view the same canvas live.
- The candidate can use Google Meet/Zoom independently on the laptop.
- A refresh does not destroy the active session.
- Temporary network loss does not silently lose acknowledged work.
- Room links are not trivially enumerable.
- Errors are observable by the developer.
- Feedback can be submitted without leaving the product.
- The production canvas licensing is legally resolved.

9. Testing Strategy

Device matrix

Device / Browser	Priority	Tests
Samsung Galaxy Tab + S Pen + Chrome	P0	Stylus, pressure, palm rejection, zoom, reconnect
iPad + Apple Pencil + Safari	P0	Stylus, touch gestures, reconnect
Android phone + Chrome	P1	Responsive viewer/editor fallback
Windows/macOS Chrome	P0	Viewer/editor, keyboard, export
Safari desktop	P1	Viewer/editor
Firefox desktop	P1	Viewer/editor

Automated tests

- Room authorization tests.
- Operation ordering/idempotency tests.
- Reconnect and replay tests.
- Snapshot restore tests.
- Rate-limit tests.
- Viewer cannot submit editor-only operations.
- Room expiration/deletion tests.
- Export correctness tests.

Manual acceptance test

```
1. Laptop creates room.
2. Tablet opens room.
3. Draw 50 strokes.
4. Viewer opens link.
5. Verify viewer sees all 50 strokes.
6. Continue drawing while viewer watches.
7. Disable tablet network for 15 seconds.
8. Restore network.
9. Continue drawing.
10. Refresh viewer.
11. Confirm final state is correct.
12. Export PNG/PDF.
13. Submit feedback.
14. Verify telemetry and errors contain no canvas-sensitive payloads.
```

10. Cost Estimation

Costs below are planning estimates based on currently published pricing where available. Actual cost depends on traffic, session duration, storage, email/analytics choices, taxes, and the canvas SDK license. USD is used for vendor comparison; INR values use a planning assumption of ■85/USD and are intentionally approximate.

Published infrastructure references

- Cloudflare Workers Paid currently starts at $5/month and includes a monthly allocation for Workers/Durable Objects; Durable Objects have usage-based request, compute, and storage billing. ■cite■turn0search0■turn0search1■
- Supabase Free is $0; Pro is currently $25/month, with included database/storage/egress allowances and usage-based additions. ■cite■turn0search2■
- Vercel Hobby is $0 and Pro is currently $20/month with included usage credit; taxes and additional usage may apply. ■cite■turn0search12■
- tldraw currently offers a 100-day trial; commercial production use requires an appropriate license and startup pricing may be available. Exact commercial pricing is not publicly fixed on the pricing page. ■cite■turn0search3■turn0search6■

Scenario A — Private development / prototype

Item	Monthly estimate	Notes
Frontend hosting	$0	Vercel Hobby or equivalent for low-traffic prototype.
Realtime	$0	Cloudflare free limits may be sufficient during early testing.
Database	$0	Supabase Free for small development workload.
Storage	$0	Keep exports minimal during prototype.
Monitoring	$0	Free tiers / basic logs.
Domain	~$1–2/mo equivalent	Annual domain purchase; depends on TLD.
Canvas SDK	$0 during trial	Must resolve license before commercial production.
Total	~$1–10/mo	Excludes your own development time.

Scenario B — Small public beta

Item	Planning estimate	Notes
Frontend	$0–20/mo	Hobby may suffice; Pro if required by workflow/features.
Realtime	$5–15/mo	Cloudflare paid baseline plus modest usage; actual usage varies.
Database	$0–25/mo	Free may work initially; Pro gives backups/higher limits.
Storage	$0–5/mo	Depends on exports and retention.
Monitoring/analytics	$0–20/mo	Free tiers likely enough initially.
Email	$0–10/mo	Only if transactional email is introduced.
Domain	~$1–2/mo equivalent	Annualized.
Canvas license	Variable	Commercial tldraw license requires quote; startup pricing may be available.
Total before canvas license	~$6–97/mo	Broad range because many services can remain on free tiers.

Scenario C — Early traction

Item	Planning range / month	Trigger
Frontend/compute	$20–75	Higher builds/traffic/server functions.
Realtime	$15–100+	Longer sessions, many concurrent rooms, more messages.
Database	$25–100+	Persistent accounts, session metadata, analytics.
Storage/egress	$5–50+	Many saved/exported sessions.
Monitoring/analytics	$10–100+	Higher event volume / paid observability.
Email/auth extras	$0–50+	Depends on product expansion.
Canvas commercial license	Variable	Must be negotiated/quoted.
Total	~$75–475+ / month	Planning envelope, not a vendor quote.

Approximate INR conversion

Monthly USD	At ■85/USD	Interpretation
$10	■850	Tiny beta
$25	■2,125	Single paid infrastructure component
$50	■4,250	Comfortable small beta
$100	■8,500	Early production usage
$250	■21,250	Meaningful traction
$500	■42,500	Larger early-stage workload

Important: the canvas license is the biggest unknown in the chosen architecture. If the commercial tldraw quote is not attractive, the project should evaluate alternative canvas libraries or a self-built canvas layer before committing to a paid public launch.

11. Basic Capacity & Cost Reasoning

Realtime traffic is likely to be modest if semantic operations are sent instead of video. A handwriting stroke may contain dozens to hundreds of points, but even a sustained drawing session is typically much smaller than sending a screen-share video stream.

For planning, use these load assumptions during beta:

Parameter	Conservative planning assumption
Average active room	2 connected clients
Average session	30–60 minutes
Viewer count	1–3 per room
Canvas operations	Hundreds to several thousand per session
Persistence	Periodic snapshots + operation checkpointing
Room retention	7–30 days for anonymous rooms
Export	User-triggered; not automatic video export

The important scaling variable is not simply registered users. It is **concurrent connected rooms × average connected duration × operation rate**. Cloudflare Durable Objects are designed for coordination among multiple WebSocket clients in a room, and Cloudflare documents WebSocket hibernation as the recommended approach to reduce duration costs during inactivity. ■cite■turn0search8■turn0search9■

12. Product Feedback & Learning System

Because the market already has collaborative whiteboards, product learning should be a core part of V1 rather than an afterthought.

In-product feedback

- Floating **Feedback** button.
- Post-session 1–5 rating.
- Category: latency / stylus / sharing / UI / missing feature / bug / other.
- Optional text description.
- Optional room/session ID automatically attached.
- Never automatically attach canvas contents to feedback.

Events to track

Event	Purpose
room_created	Activation
editor_connected	Tablet/device success
viewer_joined	Sharing success
first_stroke	Core activation
operation_sync_error	Realtime reliability
reconnect	Network reliability
session_ended	Session duration
export_clicked	Artifact value
feedback_submitted	Product learning

Core product metrics

- **Activation:** % of rooms that produce a first stroke.
- **Share success:** % of rooms with at least one viewer.
- **Realtime success:** % of operations acknowledged/rendered without error.
- **Repeat usage:** users/rooms returning within 7 and 30 days.
- **Session quality:** median session duration and disconnect rate.
- **Product pull:** percentage of users who voluntarily submit feedback or create another room.

13. Security, Abuse & Privacy

A public anonymous collaboration product can be abused. V1 needs basic controls even before it has a large audience.

- Random, high-entropy room identifiers.
- Signed editor/viewer capabilities.
- Connection and room-creation rate limits.
- Maximum room/session lifetime for anonymous rooms.
- Maximum canvas/object size to prevent memory abuse.
- Payload-size limits on WebSocket messages.
- Server-side validation of operation types.
- Basic IP/device abuse throttling with privacy-conscious retention.
- Report/abuse contact.
- Security headers and HTTPS.
- Dependency scanning and secret management.

Threats to explicitly consider

Threat	Mitigation
Room enumeration	High-entropy IDs; never sequential.
Unauthorized editing	Separate editor token from viewer token; validate server-side.
WebSocket flooding	Rate limits + payload limits + connection limits.
Canvas memory exhaustion	Object/stroke quotas and snapshot size limits.
Stored malicious content	Sanitize text/HTML; do not render arbitrary HTML.
Sensitive interview information	Minimal retention; clear privacy policy; no audio/video V1.
Abuse/spam	Rate limits, report mechanism, expiry.

14. AI/Coding-Agent Handoff Specification

If this PDF is provided to an AI coding agent, the agent should interpret the project as follows:

```
PROJECT: Live Tablet Whiteboard

GOAL
Build a production web app where a user can create a realtime
whiteboard room, open the room on a tablet with a stylus, draw/write,
and have another browser see the canvas live through a shareable URL.

PRIMARY V1 USER
A technical interview candidate using a laptop for Google Meet/Zoom
and a tablet as the visual explanation surface.

MANDATORY V1
- Browser-first; no native app required.
- Anonymous room creation.
- Shareable viewer URL.
- Editor and read-only viewer roles.
- Stylus/touch canvas.
- Pen, eraser, select, arrow/line, rectangle, text, undo/redo.
- Near-realtime synchronization.
- Reconnect handling.
- Persistence / restore.
- QR pairing.
- Feedback.
- Basic analytics.
- Production security and rate limiting.
- Deployment with HTTPS.

DO NOT BUILD IN V1
- Video/audio calls.
- AI features.
- Native apps.
- Enterprise accounts.
- Payments.
- Complex diagram automation.
- Full interview platform.

ARCHITECTURE PREFERENCE
- Next.js + TypeScript frontend.
- Production-licensed canvas SDK (tldraw is a candidate).
- Cloudflare Workers + Durable Objects + WebSockets for room realtime.
- Supabase Postgres for persistent metadata where appropriate.
- Vercel/Cloudflare for web deployment.
- Sentry + product analytics.

CORE DATA FLOW
Tablet editor
 -> canvas operation
 -> WebSocket
 -> room coordinator
 -> broadcast operation
 -> viewer browser
 -> render

PERSISTENCE
Periodically checkpoint/snapshot the room state so a refresh/reconnect
can restore the canvas.

SECURITY
Viewer must not be able to edit.
Room IDs must be non-enumerable.
All room operations must be validated server-side.
Apply rate/payload/connection limits.

SUCCESS CRITERIA
A candidate can create a room, open it on a Samsung tablet with S Pen,
draw while on a laptop video call, send the viewer URL, and the
interviewer can watch the drawing live with no installation.
```

Agent execution rule

An implementation agent should first produce the repository architecture, environment variables, database schema, room protocol, and local development instructions. It should then implement the smallest vertical slice: create room → open editor → connect viewer → draw one stroke → synchronize → refresh → restore. Only after that path is working should it add the rest of the tools and polish.

15. Post-V1 Roadmap

Version	Features	Why
V1.1	Better tablet UX, follow presenter, improved export, room history	Improve core workflow
V1.2	Equation recognition, handwriting-to-LaTeX	Strong mathematics/technical use case
V1.3	AI diagram cleanup and architecture understanding	Differentiate beyond whiteboards
V1.4	Interview templates: system design, DSA, ML, math	Targeted workflows
V1.5	Session replay + notes	Turn sessions into reusable artifacts
V2	Accounts, saved workspaces, collaboration, sharing permissions	Retention and power users

Potential AI layer

- Convert handwritten equations into LaTeX on demand.
- Recognize boxes/arrows/labels and produce a clean architecture diagram.
- Explain the user's diagram.
- Identify missing components or possible bottlenecks as suggestions, not automatic edits.
- Generate interview notes from the completed canvas.
- Create a polished PDF/Markdown summary after the session.

Important product principle

AI should initially be **on-demand**, not constantly interpreting every stroke. Continuous AI interpretation adds latency, cost, privacy concerns, and distraction. The core realtime canvas should remain useful with zero AI.

16. Public Beta Launch Checklist

Area	Checklist
Product	Create room; share link; tablet editor; viewer; tools; persistence; export; feedback.
Realtime	Reconnect; latency telemetry; operation validation; multi-viewer test.
Security	HTTPS; signed capabilities; rate limits; payload limits; room expiry.
Privacy	Privacy policy; data retention statement; deletion/expiry behavior.
Legal	Terms of use; canvas SDK production license; third-party licenses.
Operations	Error monitoring; uptime check; logs; alerts; backups where applicable.
Analytics	Activation funnel; viewer join; sync failures; retention.
Devices	Samsung/S Pen; iPad/Pencil; desktop Chrome; Safari; Firefox.
Feedback	In-product feedback; issue reporting; public roadmap or changelog.

Launch strategy

- Start with an invite-free public beta but impose conservative anonymous room limits.
- Target technical interview candidates first because the use case is easy to explain.
- Post a short demonstration showing laptop Meet + tablet drawing + interviewer browser.
- Measure whether users return, not only whether they create one room.
- Interview the first 20–50 active users and build the next features from observed friction.

17. Final Build Recommendation

Build it as a real public beta, but keep V1 aggressively focused.

The strongest version of the idea is not “an online whiteboard.” It is **“turn your tablet into a live visual-thinking second screen.”** The interview use case is the first wedge because the workflow is concrete and easy to test. Teaching, technical collaboration, architecture explanation, and mathematical problem solving can become later markets.

The technical foundation is straightforward enough for a side project: a browser canvas, a room-oriented WebSocket service, persistence, and a very simple sharing model. The harder questions are product quality, stylus UX, perceived latency, and whether people prefer it to existing whiteboards.

Recommended first milestone

```
MILESTONE 1

Create room
   ↓
Open on tablet
   ↓
Draw one stroke
   ↓
Viewer receives it live
   ↓
Refresh viewer
   ↓
Stroke is still there

If this loop is fast, reliable and pleasant,
the rest of V1 is worth building.
```

Cost conclusion

A small public beta can plausibly operate on a low monthly infrastructure budget before meaningful traction, because the realtime workload is lightweight compared with video streaming. Current published baseline prices include Cloudflare Workers Paid from $5/month, Supabase Pro at $25/month if needed, and Vercel Pro at $20/month if needed. The major unknown is the commercial canvas SDK license if tldraw is selected.
■cite■turn0search0■turn0search2■turn0search12■turn0search3■

The project should therefore budget **approximately $10–$100/month for early infrastructure before the canvas license**, then add the actual commercial canvas license and usage as required. In INR, using a planning conversion of ■85/USD, that is roughly ■850–■8,500/month. This is a planning estimate, not a vendor quote.

The next engineering action is not to build every feature. It is to prove the end-to-end realtime vertical slice on an actual Samsung tablet/S Pen and two independent browsers, then deploy that slice publicly and collect feedback.

Sources used for current pricing/licensing

Cloudflare Workers/Durable Objects pricing and WebSocket documentation; Supabase pricing; Vercel pricing; tldraw pricing and license documentation. Pricing should be rechecked immediately before committing to a paid architecture because vendor prices and license terms can change.